ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/378962192

The Impact of Artificial Intelligence on Programmer Productivity

Conference Paper · February 2024

CITATIONS
5

READS
6,081

1 author:

Ridi Ferdiana
Universitas Gadjah Mada
232 PUBLICATIONS **1,709** CITATIONS

SEE PROFILE

All content following this page was uploaded by Ridi Ferdiana on 14 March 2024.

The user has requested enhancement of the downloaded file.

KEYNOTE SPEAKER

Ridi Ferdiana

Universitas Gadjah Mada

Profile

Having an interest in the software world, Ridi started his early career as a professional programmer in Java and smartphone technology. In 2009, Ridi worked to teach many companies as a Microsoft Certified Trainer. He finished his doctoral degree at 26 by proposing a software engineering framework, namely Global eXtreme Programming (GXP). Enjoying so much in teaching and sharing, Ridi is one of the Amazon AWS educators and AWS Freelance instructors. Ridi becomes a professor at 39 years old in the software engineering field. Nowadays, he has built a framework, namely DiSiAI (digital sibling of AI), Lean Software Modeling, and Indonesia regional lead for Microsoft Certified Trainer and Microsoft MVP veteran for more than 17 years in Visual Studio developer tools.

Title:

The Impact of Artificial Intelligence on Programmer Productivity

Introduction

In the past, we built software by creating codes line by line. Therefore, one of the most valuable software metrics is a line of codes. Artificial Intelligence (AI) has become pivotal in building modern software. In modern software engineering, the rich software engineering process substantially relies on AI-supported models, e.g., Cognitive Services, Generative AI, and the recent Open AI API. Hence, cloud computing and micro-services have become pillars for supporting the modern software development lifecycle. A programmer works on the codes by generating the codes through boilerplate or template, copying the codes through samples and live documentation, and asking AI to recommend or create the codes. In other words, classical software development is dead, and AI empowers most software development processes.

Through the use of advanced AI-powered tools and techniques, developers can streamline and optimize tasks such as code generation, debugging, and optimization [1]. The benefits of incorporating AI into programming activities are significant, not only in terms of increased productivity but also in problem-solving and decision-making. By analyzing large volumes of data and identifying patterns, AI can assist developers in identifying optimal solutions. AI software such as code completion and suggestion tools, automated testing frameworks, and code refactoring

tools enhance developer productivity. The integration of AI into software development offers numerous advantages. Firstly, AI can reduce the time spent on repetitive tasks such as writing boilerplate code or performing code reviews. Secondly, AI can provide intelligent suggestions and recommendations to help developers write cleaner and more efficient code. Lastly, automating specific tasks using AI frees up developers' time to focus on more complex and creative aspects of software development. In this article, we will answer several key questions such as:

1. How can AI improve developer productivity in programming activities?
2. What scenarios can AI be applied in software development to enhance productivity?
3. What are the potential productivity gains from using AI in programming activities?

Benefits of AI in Programming Activity

Programming activities are complex and time-consuming, requiring developers to write, test, and maintain code. These activities need specific skills and knowledge, such as coding syntax, data structures, algorithms, design patterns, and debugging skills. AI can help developers in several ways to improve their productivity in programming activities:

1. AI can automate repetitive tasks such as writing boilerplate code, freeing developers' time to focus on more critical aspects of software development.
2. AI can assist in code generation, suggesting code snippets or even entire functions based on the context or requirements provided by the developer.
3. AI can provide intelligent code suggestions and recommendations, helping developers write cleaner and more efficient code.
4. AI can refactor the codes to improve code quality and maintainability.
5. AI can create automated tests and perform code reviews, helping developers quickly and accurately identify bugs or issues in their code.

Several popular AI-powered tools are available to developers, such as GitHub Copilot, Microsoft IntelliCode, AWS CodeWhisperer, and Tabnine. These tools can significantly boost developer productivity by providing code completion and suggestions, automating testing processes, assisting with code refactoring, and streamlining software. Although the cost varies depending on the specific tools and services being used, the potential productivity gains from utilizing AI in programming activities are significant. For example, GitHub Copilot offers a subscription-based pricing model with monthly plans ranging from $3 to $200 per user. The critical question is whether the subscription cost is worth the increased productivity. Research suggests that utilizing AI in programming activities can lead to productivity gains ranging from 10% to 50% [1] but only 30.5% codes is used by the programmer [2].

Quantifying the Productivity Gains from AI in Programming Activities

Quantifying the exact productivity gains from AI in programming activities can be challenging as it depends on various factors such as the specific AI tools and techniques used, the skill level and experience of the developers, the complexity of the software being developed, and the particular tasks being automated or assisted by AI. However, studies have shown that AI can significantly improve developer productivity. For example, research has shown that developers using AI tools

for code generation and suggestions can complete tasks faster and with fewer errors than manual coding. Software developers can complete coding tasks up to twice as fast with generative AI[1]. However, it is essential to note that the effectiveness of AI tools in improving developer productivity can vary depending on the context and the individual developer's skills and experience. The research method to measure programmer productivity in the context of AI tools in programming activities typically involves comparing the time and accuracy of completing tasks with and without AI. This is done by conducting experiments or studies where developers are given similar programming tasks, some using AI tools and others relying on traditional methods.

Prather et al. show that using Copilot, an AI code generator, resulted in a shift from writing code to understanding code [3]. Developers spent more time reviewing code than actually writing code [4]. In the context of AI programming assistants like GitHub Copilot, studies have shown that developers perceive themselves to be more productive when accepting suggestions from the AI tool. This perception of increased productivity is supported by metrics such as acceptance rate, contribution speed, and volume. The research suggests that AI can significantly improve developer productivity in programming activities. From simple programming problems to complex software development projects, AI tools can automate repetitive tasks, provide intelligent code suggestions, detect and fix errors, and assist in code refactoring and optimization.

Moreover, a separate study titled "Assessing the Quality of GitHub Copilot’s Code Generation" by Burak Yetistiren, Isik Ozsoy, and Eray Tüzün, which focuses on the systematic evaluation of GitHub Copilot's code suggestions, found that GitHub Copilot was able to generate valid code with a 91.5% success rate. However, out of 164 problems evaluated, 47 were correctly solved, 84 were partially correct, and 33 were incorrect. These results suggest that while GitHub Copilot shows promise as a code generation tool, it is not without its limitations and inaccuracies, and further assessment is needed [5].

Programmer Interaction with AI

Recent studies have shown that programmers interact with AI-based code generation tools like GitHub Copilot in various ways [6].

- Exploratory Interactions: Programmers use the tool for exploration, especially when unsure how to start a task or encounter code that doesn’t work as expected. They engage with the model to explore various solutions or to fix bugs.
- Prompting with Comments: Programmers guide the code-generating model using natural language comments. These comments are explicit prompts that convey the programmer's intent more clearly than the code alone. This method is found to offer programmers a greater sense of control.
- Trust and Excitement: The willingness to explore and trust in the model correlates with a programmer's excitement about the tool. However, over-reliance on the model can sometimes hinder progress by creating a gap between expectations and the model's actual capabilities.
- Different Commenting Strategies: Programmers write comments aimed at the code-generating model differently than they would for human readers. These comments may

include more detail and specific information to guide the model. After the interaction, these comments are often removed as they don't add value to the final code
- Drifting and Shepherding: New interaction patterns, such as drifting and shepherding, have been observed, where programmers either follow the suggestions of the model passively or actively guide the model toward the desired outcome.

These interactions highlight the role code-generating models play in software development, acting not just as a coding assistant but also as a discovery and educational tool, allowing programmers to experiment and learn new coding practices. However, this activity will cause the developer to miss some fundamental aspects not highlighted in code generation. Furthermore, the developer might miss some critical aspects of programming language and concepts, such as object-oriented programming, design patterns, or even essential debugging skills. Some novice developers are interested in creating a code generation result better by understanding the prompt engineering technique.

Developers are interested in prompt engineering in GitHub Copilot because it will be easier than learning programming from scratch. Prompt engineering allows developers to instruct the AI to do the requested tasks. This involves carefully selecting terminology, arranging word order, and establishing effective interaction between the developer and GitHub Copilot to obtain the desired code generation results. The effectiveness of prompt engineering becomes the key to successful code generation. For example, out of 87 problems initially unsolved by Copilot using the verbatim prompts, 53 were successfully solved after the prompts were modified using natural language [7]. An Empirical Study on GitHub Copilot" was that approximately 46% of the time, when provided with semantically equivalent but differently worded method descriptions, GitHub Copilot produced different code recommendations [8]. Therefore, prompt engineering will make code helpful generation. The study concluded that while GitHub Copilot can increase productivity as measured by the number of lines of code added, the quality of the code produced is inferior, demonstrated by having more lines of code removed in subsequent trials compared to human pair-programming. This suggests that Copilot may not be an adequate substitute for the quality assurance aspect of human pair programming [9].

Future Trends in AI-Enabled Programming Productivity

There is still a lot of agenda in software development productivity that AI empowers. Future trends in AI-enabled programming productivity include:

- Intelligent code completion and code suggestion: AI algorithms can analyze the code context, understand the developer's intent, and provide accurate recommendations for completing code snippets. AI will help with code completion and code suggestions in the future based on the developer's coding style and preference.
- Automated bug detection and debugging: AI tools can analyze code to identify potential bugs, errors, and vulnerabilities, helping developers debug their code more efficiently.
- Automated refactoring and code optimization: AI algorithms can analyze existing code and suggest improvements, such as optimizing performance, eliminating redundant code, and enhancing code. In the future, AI will pro-actively say that the codes potentially have bugs

- Automated code generation: AI models like GitHub Copilot can generate entire code blocks based on descriptive prompts, reducing the manual effort required by developers. In the future, AI will help automate code generation by seeing historical codes from teammates, the community, code comments, or documentation.
- Automated documentation generation: AI tools can analyze code and generate comprehensive and accurate documentation, making it easier for developers to understand and maintain their code. In the future, AI will help create developer documentation like JavaDoc or XML documentation without requiring developer comments.
- Automated testing and test case generation: AI algorithms can automatically generate test cases, improve code coverage, and identify potential issues in the code. In the future, AI will create unit tests based on the existing codes and simulate the code's input and output, including mockup/database generation.
- Automated code review: AI models can analyze code for adherence to coding standards and best practices and identify potential security vulnerabilities or performance issues. Incorporating AI into programming activities can significantly enhance developer productivity. AI will proactively review the codes for security, performance, and best practices in the future.

Conclusion

Based on this study, we will answer the questions that are mentioned in the Introduction section:

1. AI can significantly enhance developer productivity in programming activities by providing intelligent code completion, automated bug detection and debugging, code refactoring and optimization, and code generation.
2. AI-powered tools like GitHub Copilot offer potential benefits in software development by helping developers quickly look up the code samples, propose code generation, support code completion, identify bug detection, and refactor the codes.
3. AI tools such as GitHub Copilot can increase productivity by generating more lines of code and reaching a 91.5% success rate. The code quality may be inferior, as indicated by the study comparing Copilot with human pair programming that only solves 28% (47 issues solved correctly from 164). The productivity gain will vary between 10-50% depending on prompt engineering to AI [7].
4. Further research and improvement are needed to address the limitations and potential flaws in AI tools like GitHub Copilot, especially in the context of data analysis tasks performed by data scientist

References

[1] "Unleash developer productivity with generative AI | McKinsey." Accessed: Feb. 10, 2024. [Online]. Available: https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/unleashing-developer-productivity-with-generative-ai

[2] J. T. Liang, C. Yang, and B. A. Myers, "A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, New York, NY, USA: ACM, Feb. 2024, pp. 1–13. doi: 10.1145/3597503.3608128.

[3] J. Prather *et al.*, "'It's Weird That it Knows What I Want': Usability and Interactions with Copilot for Novice Programmers," *ACM Transactions on Computer-Human Interaction*, vol. 31, no. 1, Nov. 2023, doi: 10.1145/3617367.

[4] J. T. Liang, C. Yang, and B. A. Myers, "A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges," Mar. 2023. [Online]. Available: http://arxiv.org/abs/2303.17125

[5] B. Yetistiren, I. Ozsoy, and E. Tuzun, "Assessing the quality of GitHub copilot's code generation," in *PROMISE 2022 - Proceedings of the 18th International Conference on Predictive Models and Data Analytics in Software Engineering, co-located with ESEC/FSE 2022*, Association for Computing Machinery, Inc, Nov. 2022, pp. 62–71. doi: 10.1145/3558489.3559072.

[6] S. Barke, M. B. James, and N. Polikarpova, "Grounded Copilot: How Programmers Interact with Code-Generating Models," *Proceedings of the ACM on Programming Languages*, vol. 7, no. OOPSLA1, Apr. 2023, doi: 10.1145/3586030.

[7] P. Denny, V. Kumar, and N. Giacaman, "Conversing with Copilot: Exploring Prompt Engineering for Solving CS1 Problems Using Natural Language," in *SIGCSE 2023 - Proceedings of the 54th ACM Technical Symposium on Computer Science Education*, Association for Computing Machinery, Inc, Mar. 2023, pp. 1136–1142. doi: 10.1145/3545945.3569823.

[8] A. Mastropaolo *et al.*, "On the Robustness of Code Generation Techniques: An Empirical Study on GitHub Copilot," in *Proceedings - International Conference on Software Engineering*, IEEE Computer Society, 2023, pp. 2149–2160. doi: 10.1109/ICSE48619.2023.00181.

[9] S. Imai, "Is GitHub copilot a substitute for human pair-programming?," in *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*, New York, NY, USA: ACM, May 2022, pp. 319–321. doi: 10.1145/3510454.3522684.

View publication stats